PYTHON STRINGS

- Python strings are a sequence of characters used to handle & store textual data.
- Strings are immutable sequences of unicode characters used to store and manipulate data/text.
- They are one of the most frequently used data types.
- Very versatile for text processing, formatting, user input, file handling etc.,
- They’re fundamental data types in python that can be created with single(‘ ‘), double(“ ”) or triple(“‘ ”’) quotes.

INDEXING IN STRINGS :

String indexing refers to accessing individual characters in a string using their position or index.
A string is a sequence of characters and each character has its position number starting from 0.

There are 2 types of indexing ,
1. Positive indexing
2. Negative indexing

STRING INDEXING :

1. Positive indexing :

- Starts from (0)
- Counts from left to right

B	E	N	G	A	L	U	R	U
0	1	2	3	4	5	6	7	8

Eg. Word=BENGALURU

```
print(word(2))
print(word(8))        output=N
                                U
```

2. Negative indexing :

- Starts from (-1)
- Counts from right to left

C	L	A	S	S	R	O	O	M
-9	-8	-7	-6	-5	-4	-3	-2	-1

Eg. Word=CLASSROOM

```
print(word(-4))
print(word(-8))      output=R
                              L
```

STRING SLICING :

Simply means extracting a part of the string using indexes.

The extracted part may also be called as substring.

Colon symbols must be used.

And only square brackets should be used.

Eg. Text=“Hello,python”

```
print(text[3:10])
```

Output=lo,pyth

STRING SLICING :

Slicing from the start

Eg. text=“Hello,world”

print(text[:5])

output=Hello

Slicing to the end

Eg. text=“Hello,world”

print(text[3:])

output=lo,world

COUNTING CHARACTERS :

Counting characters in python is a fundamental task that can mean differently at different circumstances.

1.Finding length (string length)
2.Counting specific character
3.Counting every character

I. String length

To get the length of a string or to know the number of characters in a string, we can use the len() method.

Eg. a=“Hello,world”

```
print(len(a))
```

output=11

II. Counting specific characters

To know how many times a specific letter of a character appears, we need to use the .count() method.

Eg. text=“Mississippi”

```
print(text.count(“p”))
print(text.count(“i”))
```

output=2

4

III. Counting every character

To know how many times each character has appeared, we need to use the counter method.

Eg. text=“banana”

```
counts=Counter(text)

print(counts)
```

output=Counter({‘a’:3, ‘n’:2, ‘b’:1})

STRING CHECK :

To check if a certain phrase or character is present in a string, we can use the keyword in.

Eg. text=“The best things in life are never free”

print(“free” in text) output=true

Check if not

To check if a certain phrase or character is not present in a string, the keyword not in can be used.

Eg. text=“The best things in life are never free”

print(“expensive” not in text) output=true

CHANGING CASES :

Python provides several built in string methods to change the casing of characters in a string

1. .upper()
2. .lower()
3. .capitalize()
4. .title()
5. .swapcase()

1. .upper()

Changes all characters to uppercase

Eg. text="hello"

```
text=text.upper()
print(text)
```

output=HELLO

2. .lower()

Changes all characters to lowercase

Eg. text="HELLO"

```
text=text.lower()
print(text)
```

output=hello

3. .capitalize()

Changes first character to uppercase

Eg. text="python code"

```
text=text.capitalize()
print(text)
```

output=Python code

4. .title()

Changes first character of each word to uppercase

Eg. text="python code"

```
text=text.title()
print(text)
```

output=Python Code

5. .swapcase()

It changes lowercased characters to uppercase and vice-versa

Eg. text="PytHOn CoDE"

```
text=text.swapcase()
print(text)
```

output=pYThoN cOde

REMOVING WHITESPACES :

Whitespace is the empty space before and/or after the actual text. And we often need to remove these extra spaces.

.strip() : removes spaces from both the ends of a sentence

Eg. data=“ I like python “

print(data.strip()) output=I like python

We can remove spaces from either right end or left end individually

- .lstrip() : remove spaces from left end side
- .rstrip() : remove spaces from only right end side

REPLACING TEXT :

The .replace() method is used to replace a string with another string and also individual characters can be replaced.

Eg.

i. msg=”I like watermelon”

print(msg.replace(“watermelon”,”mango”)) output=I like mango

ii. msg=”Hello,world!”

print(msg.replace(“H”,”J”)) output=Jello,world!

SPLITTING STRINGS :

Splitting of strings is a method used to divide a string into several substrings.

It is carried out by different methods.

1. .split():

It is the primary method which is simple and versatile for most text processing tasks.

Eg. sentence="Python is very easy"

words=sentence.split()

print(words)

output=['Python','is','very','easy']

2. .splitlines():

Used for multilines strings to get new lines of strings

Eg. multiline=" " "Line 1

Line 2

Line 3" " "

print(multiline.splitlines())

output=['Line 1', 'Line 2', 'Line 3']

JOINING STRINGS :

It is just the inverse of splitting strings.

It joins the characters to form a new string.

While .split() method breaks the string into a list, the .join() method takes a list and combines it into a single string.

Eg. words=[“Python”,”is”,”fun”]

```
print(“ ”.join(words))
```

output=Pythonisfun

STRING CONCATENATION :

String concatenation is the process of joining two or more strings together to create a new one.

To concatenate or combine 2 strings, the + operator is usually used

Eg. a=“Hello”

b=“python”

c=a+” “+b

print(c) output=Hello python

```
user_name = "Alice"
# Combining a greeting with the variable
message = "Welcome back, " + user_name + "!"
print(message)
```

Output: Welcome back, Alice!

STRING FORMATTING (F-STRINGS) :

String formatting in python is the process of creating dynamic string by inserting variables, expressions or values turning it into readable form.

It replaces older method like concatenation. There are several methods, but the f-string is the more standard one as they are faster, easily readable and less prone to errors.

Eg. name=“John”

```
age=36
text=f“My name is {name}, I’m {age} years old”
print(text)                output=My name is John, I’m 36 years old


price=59
text=f“The price is {price} dollars”
print(text)                output=The price is 59 dollars
```

CHECKING START & END :

It checks if the given string starts or ends with a specific pattern.It is useful in filtering filenames, URL or cleaning data.It either gives true or false as output

Checking the start :

It uses .startswith() method

Eg. url="https://example.com"

```
print(url.startswith("https"))
print(url.startswith("html"))
```

output=True

False

Checking the end :

It uses .endswith() method

Eg. file="reports_2026.pdf"

```
print(file.endswith(".pdf"))
print(file.endswith(".docx"))
```

output=True

False

ESCAPE CHARACTERS :

To insert characters, that are illegal in a string, we can use an escape character. An example of illegal character is a double quotes inside a string that is surrounded by double quotes. It is just a backlash(\) followed by the character you want to insert.

Eg. text=“We are the so called “Vikings” from the north”

This is an erroneous code. To fix this, we can use the escape character \”

```
text=“We are the so called \“Vikings\” from the north”
print(text)
```

output=We are the so called “Vikings” from the north

Escape characters used in python :

Symbol	Name
\’	Single quote
\”	Double quote
\n	New line
\r	Carriage return
\t	Tab
\b	Backspace

Application of Strings in Agriculture :

1) Satellite and sensor data parsing:

Sensors in the soil or on the drones send back tons of data. These are often sent as long strings that need to be split/sliced & joined to read specific values like moisture, temperature etc.,

2) Genomic analysis and crop breeding:

In modern day crop science, DNA sequences are represented as long strings of characters(A,T,G,C) and are used to store these data.

3) Automated labelling and packaging:

In food processing industries, strings are used to generate dynamic labels. This label includes manufacture date, batch number, bar codes, expiry date and other details.

4) Livestock identification and tracking:

Every animal in a modern farm has a unique I'd, often stored as strings. This I'd links to database containing the animal's health record, age & breed.

5) Natural Language Processing (NLP):

Python strings enable agribots & advisory apps to communicate with farmers in their local language. They're helpful for disease diagnosis and analysing market trends.

6) Pest and disease diagnosis:

Agriculture AI models use strings to classify images when camera identifies a pest, it returns a string ’label’ that triggers a specific action like a targeted spray or a remedy action.

7) Market pricing and information systems:

Farmers use SMS based services or apps to check the daily crop prices. These systems take a string query like the name of a market and return it with a string coded with the price.

Eg. A farmer sends “Price of Ragi in Bangalore” and receives a formatted string back with the current rate per quintal or Kg.

THANK YOU